

LEVEL 4 — DEVELOPMENT TRACK CAPSTONE

Research Infrastructure

The shared platform that makes research fast, reproducible and safely promotable to production

Covers: Research environments · data pipelines · feature stores · backtesting frameworks · experiment tracking · configuration management · reproducibility · model registry · results storage · monitoring/logging · research-to-production pipelines

Table of Contents

Right-click and choose “Update Field” (or “Update entire table”) after opening in Word to populate page numbers.

Table of Contents	2
14. Research Infrastructure (Level 4)	3
14.1 Learning Position at a Glance	3
14.2 Research Environments	4
14.3 Data Pipelines.....	4
14.4 Feature Stores	5
14.5 Backtesting Frameworks.....	5
14.6 Experiment Tracking	6
14.7 Configuration Management	6
14.8 Reproducibility	7
14.9 Model Registry	7
14.10 Results Storage.....	7
14.11 Monitoring & Logging	7
14.12 Research-to-Production Pipelines.....	7
14.13 Worked Exercise — Model Answer	8
14.14 Common Mistakes — Deeper Look.....	9
14.15 Advanced Extensions	9
14.16 Professional Applications	10

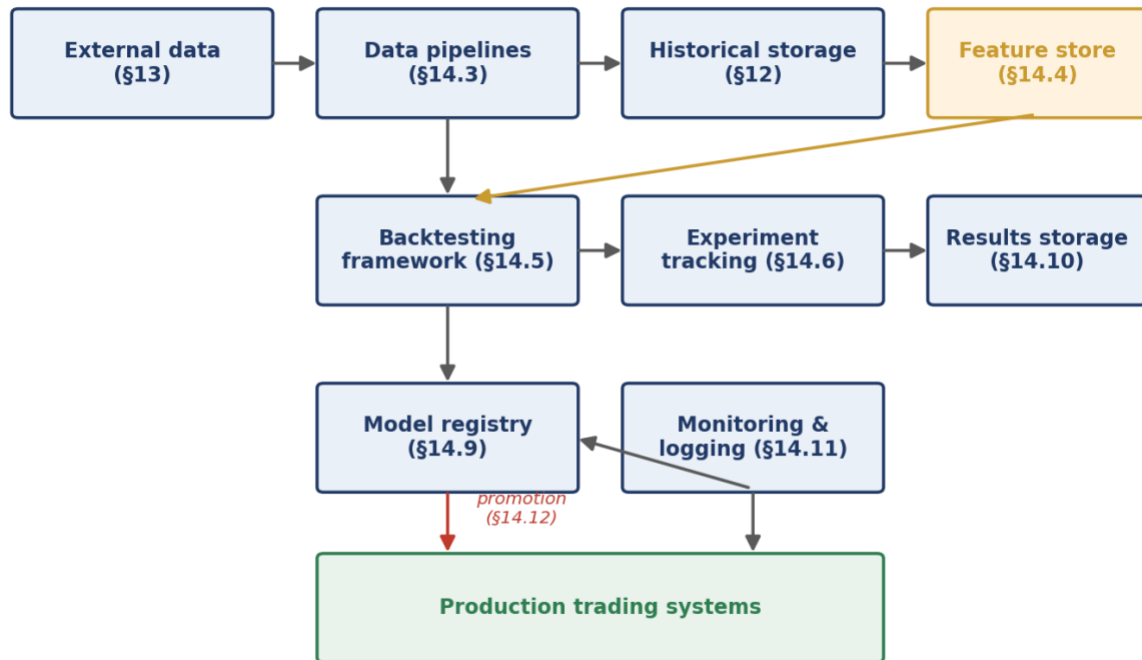
14. Research Infrastructure (Level 4)

Covers: Research environments · data pipelines · feature stores · backtesting frameworks · experiment tracking · configuration management · reproducibility · model registry · results storage · monitoring/logging · research-to-production pipelines

14.1 Learning Position at a Glance

- **What it is:** The shared platform that makes research fast, reproducible and safely promotable to production.
- **Why it matters:** Without this, every researcher rebuilds their own ad hoc backtester and feature pipeline, results are irreproducible, and nothing can be safely promoted to live trading.
- **Prerequisites:** Software Engineering Foundations; Databases; Testing; Backtesting (Level 3).
- **Connects to Research:** A shared, tested backtesting framework and feature store is what lets a research team compare strategies apples-to-apples.
- **Connects to Development:** This is the centrepiece Level 4 deliverable — it is what separates a development track from ad hoc scripting.
- **Connects to Trading:** A clean research-to-production pipeline is what prevents ‘it worked in the notebook but not live’ failures.
- **Practical exercise:** Design (and implement a minimal version of) a feature store that computes and caches a set of features once, reusable across multiple strategy backtests.
- **Common beginner mistakes:**
 - Every researcher maintaining their own copy of data-cleaning and feature logic, leading to inconsistent results across the team.
 - No experiment tracking, making it impossible to reproduce a result from three months ago.
- **Advanced extensions:** Full model registry and automated research-to-production promotion pipelines (Level 5–6).
- **Professional applications:** Team-scale research platforms; reproducible strategy development.

The research infrastructure platform, end to end



Research environment (§14.2) and configuration management (§14.7) wrap the whole platform; reproducibility (§14.8) is the property every layer above is designed to preserve.

Figure 14.1 — Research infrastructure is not one tool but a connected platform: data flows from external sources through storage and a shared feature store into backtesting, tracked experimentation, and — for strategies that clear an explicit promotion gate — into monitored production.

14.2 Research Environments

A research environment is the standardised, reproducible computing setup a researcher works in — a fixed set of installed libraries and versions, consistent access to data and compute, and ideally identical whether run on a laptop, a shared server, or a cloud notebook. Without standardisation, 'works on my machine' becomes a constant, low-grade source of wasted time and irreproducible results.

Example 14.1 — Pinned dependencies as the minimum standard for a shared research environment

```
# pyproject.toml (or requirements.txt) pins exact dependency versions,  
# so every researcher's environment resolves to the same library set.  
[project]  
dependencies = [  
    "pandas==2.2.1",  
    "numpy==1.26.4",  
    "scikit-learn==1.4.0",  
]  
  
# A containerised (e.g. Docker) environment goes further, pinning the  
# operating system and system libraries too, not just Python packages.
```

14.3 Data Pipelines

The ingestion, validation and storage pipelines covered in full in §13 are the entry point to the research infrastructure platform: every downstream layer — feature store, backtesting, experiment tracking — is only as trustworthy as the data pipelines feeding it. Research infrastructure does not duplicate that discipline; it depends on it.

14.4 Feature Stores

A feature store computes a research feature (a signal, a risk metric, a normalised price series) once and makes the result available for reuse by any strategy or researcher that needs it, rather than each strategy recomputing the same underlying calculation independently. Beyond avoiding redundant computation, the more important benefit is consistency: if momentum is computed once, in one place, every strategy that uses it is guaranteed to be using the exact same definition.

A feature store removes duplicated, drifting feature logic

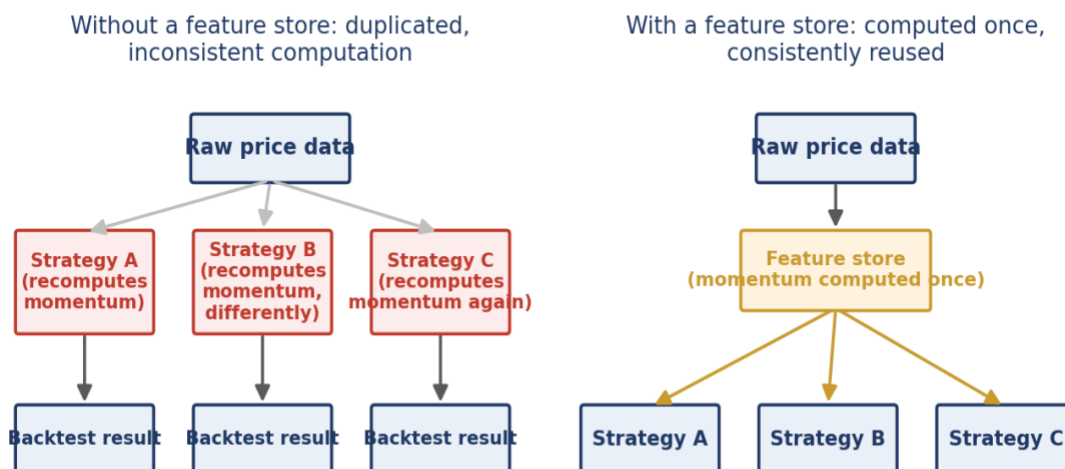


Figure 14.2 — Without a shared feature store, three strategies each reimplement 'momentum' independently, and small differences between the three implementations silently make results incomparable; a feature store computes it once and every strategy consumes the identical result.

Example 14.2 — A minimal feature store: compute once, cache, and serve from cache thereafter

```
class FeatureStore:
    def __init__(self, cache_dir: str):
        self.cache_dir = cache_dir

    def get(self, feature_name: str, universe: list[str], as_of: str) -> pd.DataFrame:
        cache_path = f"{self.cache_dir}/{feature_name}_{as_of}.parquet"
        if os.path.exists(cache_path):
            return pd.read_parquet(cache_path)[universe]

        result = self._compute(feature_name, universe, as_of)
        result.to_parquet(cache_path)
        return result

    def _compute(self, feature_name: str, universe: list[str], as_of: str) -> pd.DataFrame:
        if feature_name == "momentum_20d":
            prices = load_prices(universe, end=as_of)
            return prices.pct_change(20)
        raise ValueError(f"Unknown feature: {feature_name}")
```

14.5 Backtesting Frameworks

A shared backtesting framework — built on the Signal/Portfolio/Backtester design from §9.10 and validated per §11.6 — is what lets every strategy in a research team be evaluated on identical, tested infrastructure rather than each researcher's own slightly-different backtesting script. This is precisely what makes strategies comparable apples-to-apples: a difference in reported Sharpe ratio reflects a real difference between strategies, not a difference between two different, unvalidated backtesting implementations.

14.6 Experiment Tracking

Experiment tracking logs, for every research run, the exact code version, parameters, and resulting metrics together as a single, permanent record. Without it, a promising result from three months ago — the exact common mistake named in §14.1 — becomes nearly impossible to reproduce: which of dozens of since-modified parameters produced that specific number is simply not recorded anywhere.

Every run logged: code version, parameters, and metrics together					
run_id	git commit	signal	lookback	Sharpe	date
run_0231	a1b2c3d	momentum	20	1.18	2026-03-02
run_0232	a1b2c3d	momentum	40	0.94	2026-03-02
run_0233	f4e5d6c	reversal	5	0.71	2026-03-05
run_0234	9a8b7c6	combined	20/5	1.35	2026-03-11

Figure 14.3 — A minimal experiment log: every run's git commit (linking to §10.6's tagging discipline), parameters, and headline metric recorded together, so any past result can be traced back to the exact code and configuration that produced it.

Example 14.3 — Logging the git commit alongside every run's parameters and metrics

```
import subprocess, json, datetime

def log_experiment(run_id: str, params: dict, metrics: dict) -> None:
    commit = subprocess.check_output(
        ["git", "rev-parse", "--short", "HEAD"]).decode().strip()
    record = {
        "run_id": run_id,
        "git_commit": commit,
        "params": params,
        "metrics": metrics,
        "timestamp": datetime.datetime.utcnow().isoformat(),
    }
    with open("experiments.jsonl", "a") as f:
        f.write(json.dumps(record) + "\n")
```

14.7 Configuration Management

Extending the externalised-configuration discipline from §9.8 to the whole research platform: every backtest run should be fully described by a single, versioned configuration file (which universe, which date range, which feature parameters), not by whatever values happened to be hardcoded or manually typed into a notebook cell at the time. This is a prerequisite for experiment tracking (§14.6) to be meaningful — the config itself is exactly what should be logged alongside each run.

Example 14.4 — A single configuration file fully describing one backtest run

```
# config/momentum_v3.yaml
universe: sp500
start_date: "2015-01-01"
end_date: "2024-12-31"
signal:
  name: momentum
  lookback_days: 20
portfolio:
  starting_cash: 1000000
  max_position_pct: 0.05
```

14.8 Reproducibility

Reproducibility is the property every other layer of this platform exists to protect: given a research environment (§14.2), a code version (§10.6), a configuration (§14.7), and the data it read (§6.4, §13.8), any past result should be exactly regenerable. This directly extends the reproducibility discipline introduced in §6.5 from a single researcher's discipline into a property the whole platform guarantees by construction, rather than something that depends on any individual remembering to be careful.

14.9 Model Registry

A model registry is a central, versioned catalogue of trained models or finalised strategies that have been formally registered as candidates for production use — distinct from the many exploratory versions tried during research. Registering a strategy records exactly which code version, configuration, and training/backtest data produced it, alongside its validation results, creating a clear, auditable boundary between 'something a researcher tried' and 'something under formal consideration for live trading.'

14.10 Results Storage

Backtest and experiment results deserve the same structured, queryable storage discipline as the market data covered in §12 — a properly indexed database table of run results (per-run Sharpe ratio, drawdown, turnover, alongside the experiment metadata from §14.6) supports research questions like 'show me every momentum variant tested in the last year, ranked by Sharpe' as a simple query, rather than requiring someone to manually reconstruct results scattered across individual notebooks.

14.11 Monitoring & Logging

Once a strategy is promoted to production (§14.12), monitoring and logging — building on the structured logging discipline from §9.7 and the data validation from §11.7 — continuously track whether its live behaviour matches expectations: realised performance versus backtested expectations, data feed health, and risk-limit adherence. This is what turns the alpha-decay awareness from §6.3 into an operational practice: a live strategy's performance is watched continuously, not assumed to match its backtest indefinitely.

14.12 Research-to-Production Pipelines

A research-to-production pipeline formalises the path a strategy takes from an initial research finding to live capital, with explicit gates at each stage rather than an informal handoff. This is precisely what prevents the 'it worked in the notebook but not live' failure named in §14.1: each gate (code review, staging validation) exists specifically to catch a category of problem that a notebook environment does not surface.

A research-to-production pipeline with explicit promotion gates

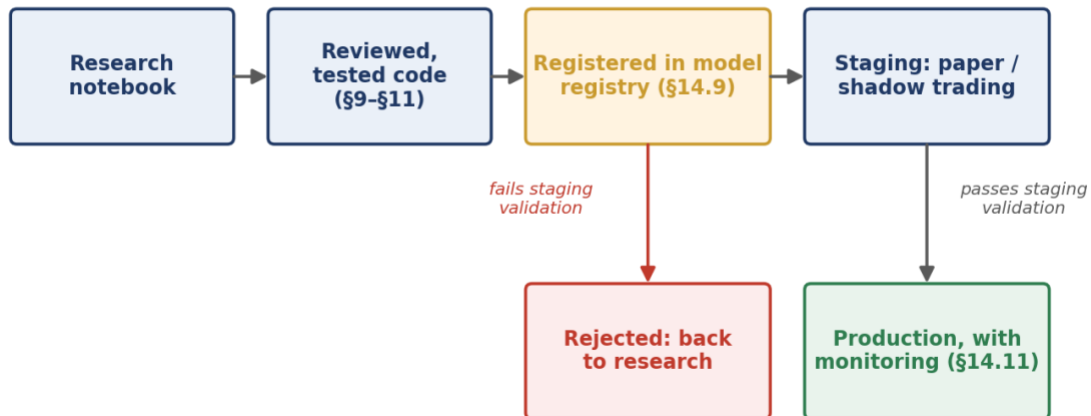


Figure 14.4 — A strategy only reaches production after its research code has been reviewed and tested (§9–§11), formally registered (§14.9), and has passed a staging period (paper or shadow trading) that validates live behaviour against backtested expectations before any real capital is committed.

- **Staging (paper/shadow trading):** Running the strategy against live data with no real capital at risk, comparing its live signals and simulated fills against what the backtest would have predicted — a direct check for the kind of research-to-production gap this section exists to prevent.
- **Explicit rejection path:** A strategy that fails staging validation returns to research with a specific, documented reason, rather than either being quietly abandoned or pushed to production anyway under time pressure.

14.13 Worked Exercise — Model Answer

Exercise: Design (and implement a minimal version of) a feature store that computes and caches a set of features once, reusable across multiple strategy backtests.

Model answer — a minimal but genuinely reusable feature store

```
import os
import pandas as pd

class FeatureStore:
    """Computes features once per (feature, universe, as_of) key and
    caches the result, so repeated requests -- from any strategy --
    reuse the same cached computation rather than recomputing it.
    """
    def __init__(self, cache_dir: str, price_loader):
        self.cache_dir = cache_dir
        self.price_loader = price_loader
        os.makedirs(cache_dir, exist_ok=True)

    def get(self, feature_name: str, universe: tuple[str, ...], as_of: str) -> pd.DataFrame:
        key = f"{feature_name}__{'-'.join(sorted(universe))}__{as_of}"
        cache_path = os.path.join(self.cache_dir, f"{key}.parquet")
        if os.path.exists(cache_path):
            logger.info("Feature cache hit: %s", key)
            return pd.read_parquet(cache_path)

        logger.info("Feature cache miss: %s -- computing", key)
        result = self._compute(feature_name, universe, as_of)
        result.to_parquet(cache_path)
        return result
```

```
def _compute(self, feature_name: str, universe, as_of: str) -> pd.DataFrame:
    prices = self.price_loader(universe, end=as_of)
    if feature_name == "momentum_20d":
        return prices.pct_change(20)
    if feature_name == "reversal_5d":
        return -prices.pct_change(5)
    raise ValueError(f"Unknown feature: {feature_name}")

# Two strategies sharing the same underlying feature computation:
store = FeatureStore(cache_dir="./feature_cache", price_loader=load_prices)
momentum = store.get("momentum_20d", universe=tuple(sp500_tickers), as_of="2024-12-31")
# A second, later call with identical arguments hits the cache instantly
# rather than recomputing -- and both strategies see the identical result.
momentum_again = store.get("momentum_20d", universe=tuple(sp500_tickers), as_of="2024-12-31")
```

This directly demonstrates the benefit illustrated in Figure 14.2: any number of strategies calling `store.get("momentum_20d", ...)` with the same arguments receive the identical cached DataFrame, computed exactly once — eliminating both the wasted recomputation and, more importantly, the risk of two strategies silently using two subtly different momentum calculations.

14.14 Common Mistakes — Deeper Look

Mistake: Every researcher maintaining their own copy of data-cleaning and feature logic, leading to inconsistent results across the team

This is the organisational-scale version of the code-duplication problems §9's SOLID principles address for a single codebase: without a shared feature store (§14.4), each researcher's slightly different momentum calculation, slightly different handling of missing data, or slightly different universe filter means that two researchers' reported Sharpe ratios for 'the same' signal are not actually comparable — the difference could reflect a real difference in the underlying idea, or it could simply reflect two different, undocumented implementations of the same idea. The cost compounds over time: as more researchers join and each brings their own conventions, comparing any two pieces of research becomes progressively less meaningful.

Mistake: No experiment tracking, making it impossible to reproduce a result from three months ago

A promising result that cannot be reproduced is, per §6.5's framing, not yet a result at all — and three months is enough time for the underlying code, data, and a researcher's own memory of the specific parameters used to all have moved on. Without the experiment log from §14.6, revisiting an old, promising number means attempting to reconstruct, from memory and scattered notebook cells, a configuration that may no longer even be fully recoverable — precisely the failure mode experiment tracking is designed to make impossible.

⚠ Case study: a research team that could not explain its own best result

- A research team's strongest-performing strategy, discovered several months earlier, was proposed for live deployment; when asked to confirm the exact configuration and data version that produced the reported number, no one on the team could reconstruct it with confidence.
- The original researcher had since modified the shared feature-computation script (with no feature store or versioning in place) for unrelated work, and no experiment log existed recording which version of that script, or which parameters, had produced the original result.
- Rebuilding the result from scratch took several weeks and produced a materially different, weaker number — strongly suggesting the original result had depended on a since-lost implementation detail, quite possibly a bug, rather than a genuine, reproducible edge; the team subsequently implemented the feature store and experiment tracking practices in §14.4 and §14.6 specifically to prevent a recurrence.

14.15 Advanced Extensions

Full model registry and automated research-to-production promotion pipelines (Level 5–6): the manual staging and review process in §14.12 can be extended into a fully automated pipeline — analogous to the CI/CD pipelines from §10.7 and §11.10, but for strategies rather than code — where passing a defined set of staging criteria automatically triggers promotion, with human sign-off required only for exceptions. This is deliberately left as an advanced extension: automating a promotion decision is only safe once the staging criteria it depends on (Figure 14.4) have themselves been proven reliable through a considerable amount of manual, supervised use.

14.16 Professional Applications

- **Team-scale research platforms:** Every practice in this section, taken together, is what allows a research team to grow past a handful of researchers without each new hire slowing the team down by introducing their own inconsistent tooling and conventions.
- **Reproducible strategy development:** The combination of experiment tracking (§14.6), configuration management (§14.7), and the underlying reproducibility discipline (§14.8, §6.5, §10.6) is what allows a strategy's entire development history — not just its final version — to be trusted, audited, and defended to a risk committee or external reviewer.